Primer entorno de desarrollo: terminal, Git y primer commit (Bloque 2) / Terminal, Git y por qué los secretos no van al repositorio

✓ **Hecho:** Ver

📘 **UNIDAD U1.2** · Setup técnico: terminal, Git, entorno Python y cliente HTTP

Terminal, Git y por qué los secretos no van al repositorio

📋 Prerrequisitos técnicos explícitos

Antes de comenzar con esta lección, asegúrate de contar con lo siguiente:

✓ **Python 3.10+** instalado en tu sistema y accesible desde la terminal.

✓ **Cuenta activa en GitHub** para alojar y respaldar tus repositorios remotos.

✓ **Visual Studio Code (VS Code)** instalado como editor principal de desarrollo.

Con el mapa del programa y tu Ficha de Alineación Técnica ya listos, empieza el trabajo técnico.

Esta lección instala los conceptos que vas a usar todos los días del programa: tu editor de código, **la terminal**, **Git**, los **entornos virtuales de Python**, y una regla que no tiene excepciones: los secretos (contraseñas, llaves de API, tokens) nunca van al repositorio.

💡 **Consejo**: cuenta con el proveedor de modelos que vas a usar en el curso (la vas a necesitar para

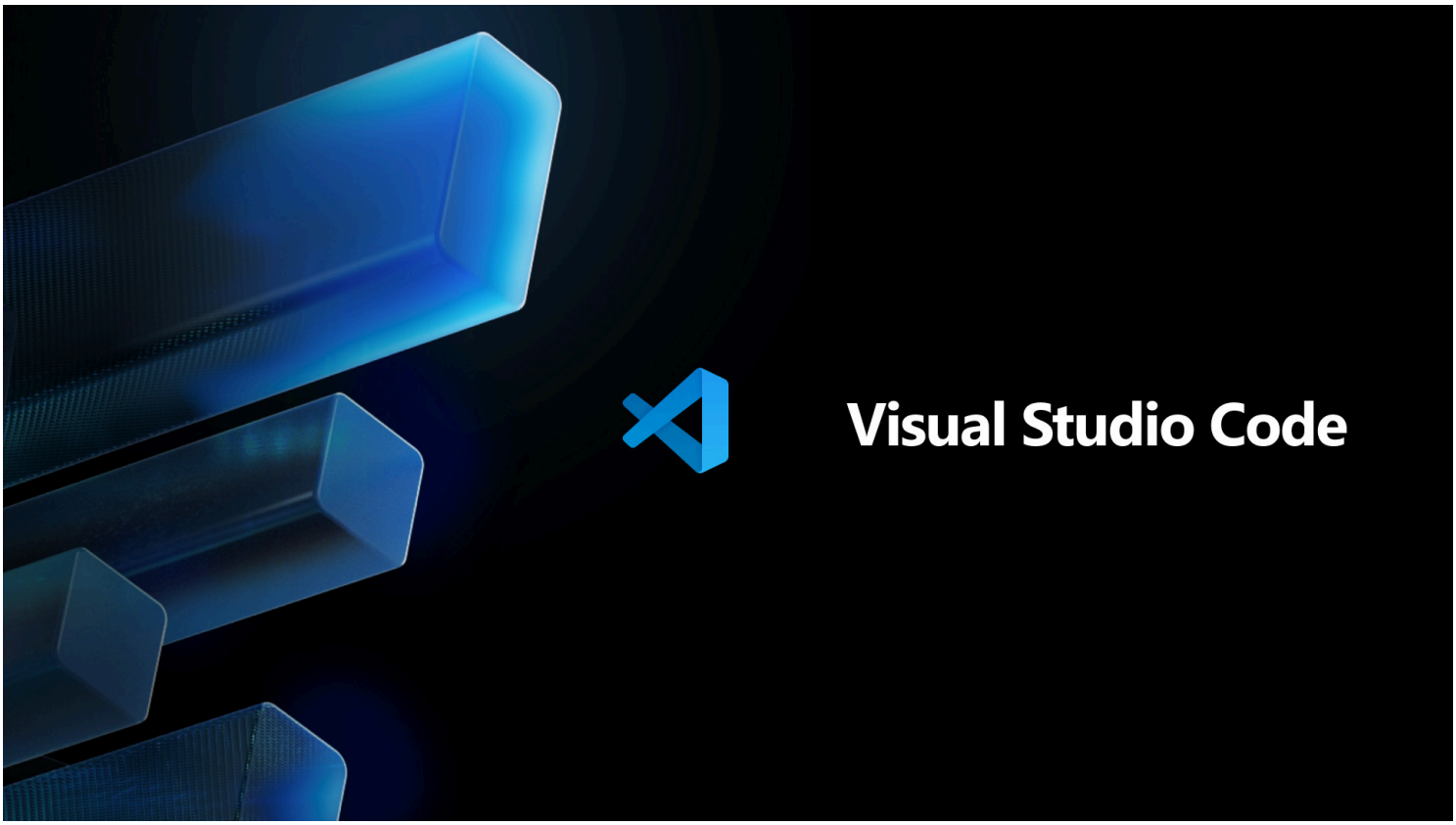
obtener tu propia API key más adelante, en el Bloque 4 de esta unidad).

> ⭐ **Recomendación**: Google Gemini API, la opción más accesible para empezar, con un nivel gratuito permanente que no pide tarjeta de crédito.

💻 Visual Studio Code: tu editor de referencia

> **Visual Studio Code (VS Code)** es el editor de código que vas a usar como referencia durante todo el programa.

La razón para elegirlo no es solo que es gratuito y liviano, sino que reúne en un mismo lugar todo lo que necesitas para trabajar sin saltar constantemente entre programas: una terminal integrada, control de Git visual además del de línea de comandos, y un explorador de archivos donde vas a crear y revisar cosas como tu archivo de dependencias o tu archivo de variables de entorno.



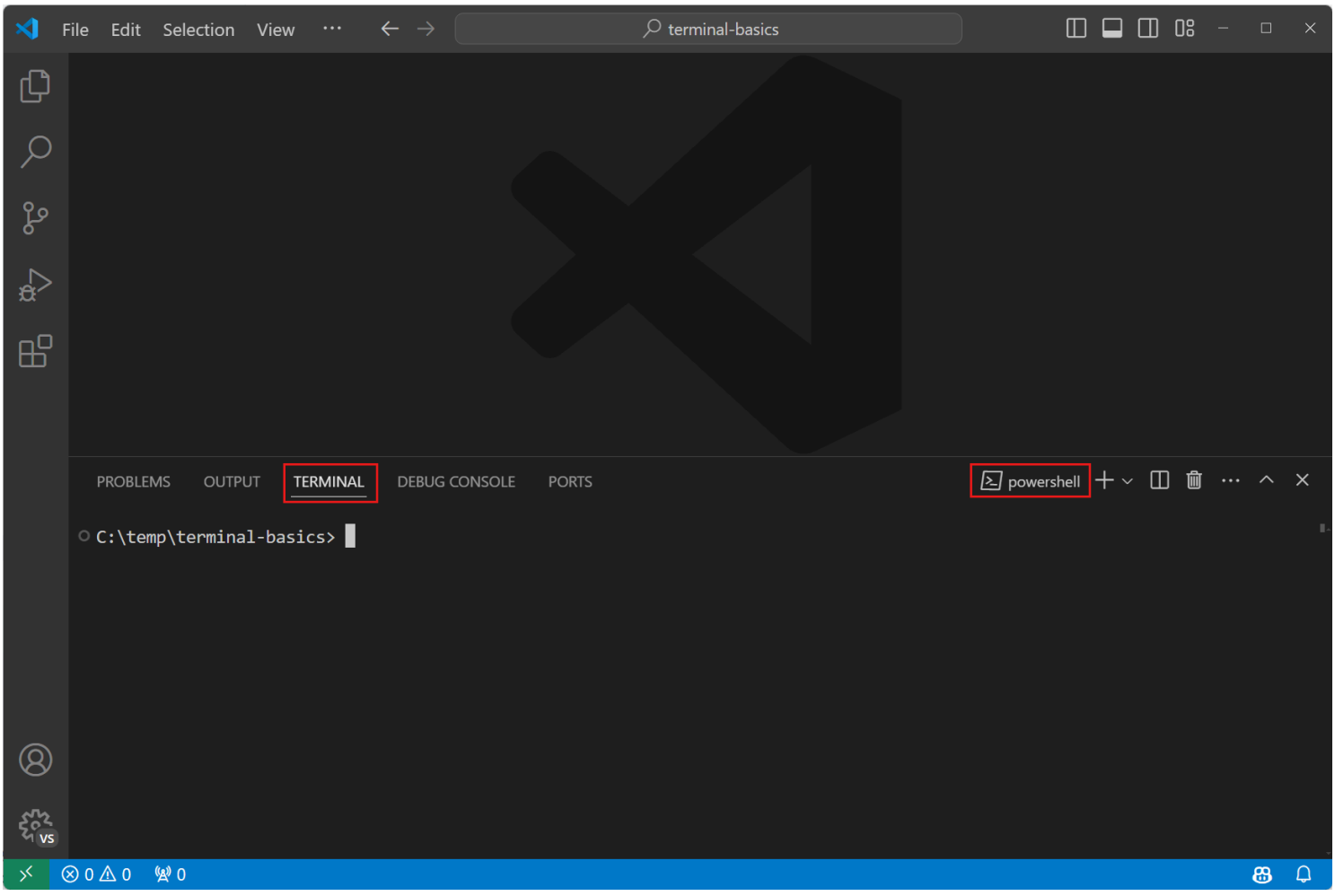
Puedes descargar **Visual Studio Code (VS Code)** desde su sitio oficial.

A lo largo de esta lección, trabajarás directamente desde **VS Code** con herramientas como la terminal, Git, los entornos virtuales y las variables de entorno.

⌨️ La terminal: tu herramienta de todos los días

> La **terminal** es la forma más directa de decirle a tu computador qué hacer, sin pasar por una interfaz gráfica.

En este programa la vas a usar constantemente para moverte entre carpetas, crear archivos, ejecutar scripts, instalar dependencias y correr comandos de Git, casi siempre desde la terminal integrada de **VS Code** para no salir del editor.



💡 **No hace falta memorizar decenas de comandos**: con navegar entre directorios, listar contenido, crear y borrar archivos y carpetas, y ejecutar un programa, ya cubres el 90% de lo que vas a necesitar en las primeras semanas.

El resto lo vas incorporando sobre la marcha, cuando lo necesites para un caso concreto.

Antes de seguir, confirma que ya tienes instalados Python y Git en tu computador (los vas a necesitar desde la próxima lección): descarga Python desde su sitio oficial y Git desde su sitio oficial si

todavía no los tienes.

Pruébalo ahora mismo: crea una carpeta de prueba, entra en ella, y confirma que tu terminal reconoce ambos comandos.

```
mkdir carpeta-de-prueba
cd carpeta-de-prueba
python --version
git --version
```

Si alguno de estos comandos no se reconoce, usa el link de instalación correspondiente más arriba. En macOS es normal que "python" no funcione y sí "python3" (lo vas a ver en detalle en la guía de troubleshooting más adelante en esta unidad). Puedes borrar esta carpeta de prueba apenas termines: no la vas a necesitar después.

🧠 Git: la memoria de tu proyecto

> **Git** es un sistema de control de versiones: guarda el historial completo de cambios de tu proyecto, para que puedas volver atrás, comparar versiones, o trabajar en paralelo sin pisar el trabajo de otra persona.

Tres conceptos bastan para empezar.



Repositorio

Un **repositorio** es la carpeta de tu proyecto ya bajo control de versiones, con su historial completo guardado.



Commit

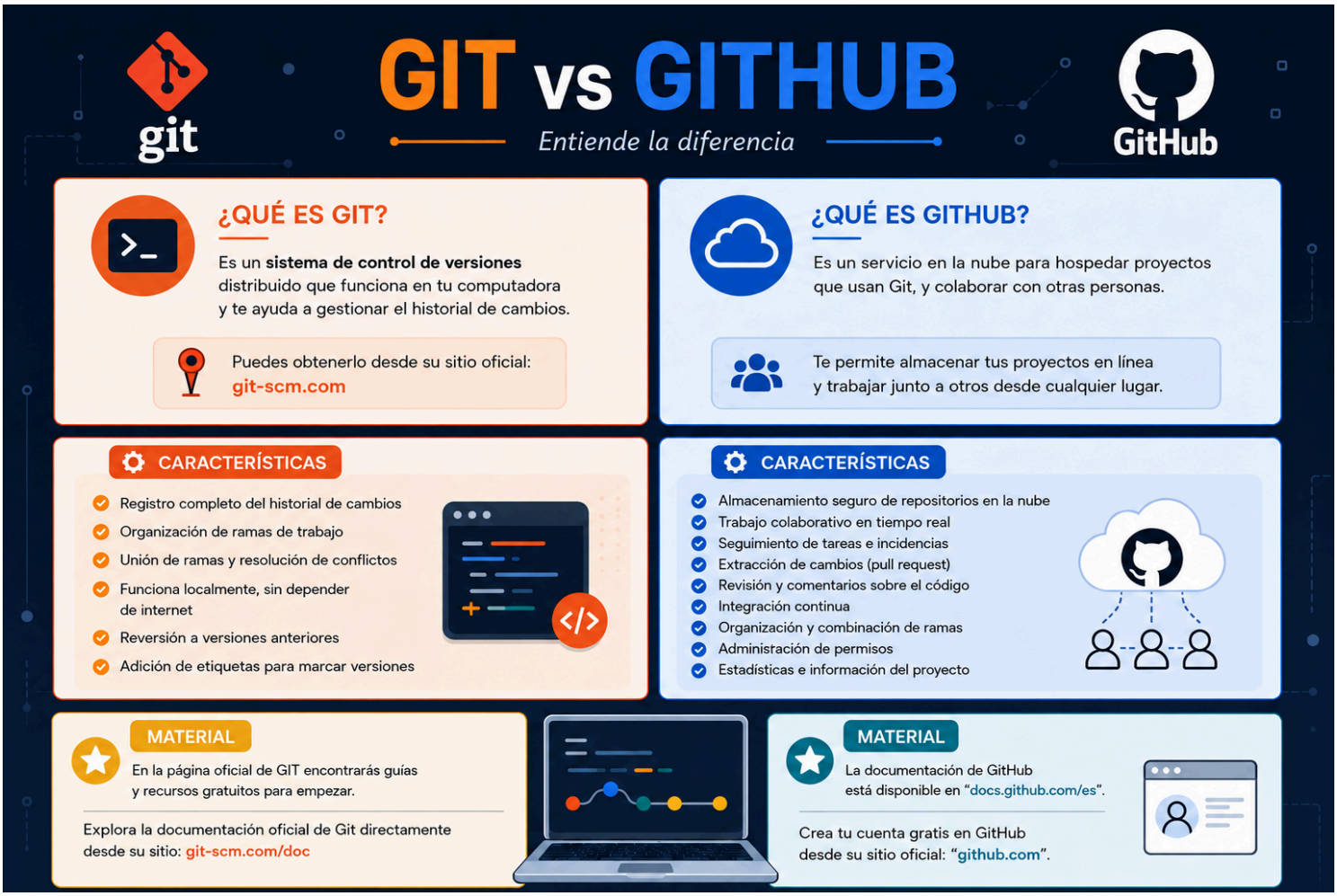
Un **commit** es una fotografía de tu proyecto en un momento dado, con un mensaje que explica qué cambió y por qué; vas a hacer decenas de commits a lo largo del programa, y cada uno debería representar un



Rama (branch)

Una **rama** (**branch**) es una línea de trabajo independiente que después puedes fusionar con la principal; en este programa vas a usarlas sobre todo para probar cosas sin arriesgar tu versión ya funcional.

cambio con sentido propio, no un batido de todo lo que hiciste en el día.



El **repositorio** vive en tu máquina, pero también en un remoto: una copia en **GitHub** que sirve como respaldo, como historial visible para el equipo docente, y como el lugar desde donde vas a entregar tu proyecto en cada **Gate**.

Todo el trabajo que vas a mostrar en este programa, sin excepción, va a vivir en un repositorio de **GitHub**.

A continuación se muestra un ejemplo práctico para inicializar un repositorio en Git y realizar tu primer commit:

```
git init
git add .
git commit -m "Initial commit: estructura inicial del proyecto"
```

Output visual esperado:

```
Initialized empty Git repository in /ruta/a/tu-proyecto/.git/
[main (root-commit) a1b2c3d] Initial commit: estructura inicial del proyecto
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
```

🧩 Entornos virtuales: cada proyecto con lo suyo

> Un **entorno virtual de Python** es una instalación aislada de Python y sus paquetes, exclusiva para un proyecto. Sin esto, instalar una librería para un proyecto puede romper silenciosamente otro proyecto que dependía de una versión distinta de esa misma librería.



La práctica en este programa es simple: cada proyecto nuevo empieza con su propio entorno virtual, y las dependencias que instalas quedan registradas en un archivo dentro del repositorio, para que cualquier otra persona (o tú mismo en otro computador) pueda reconstruir exactamente el mismo entorno con un solo comando.

Ejemplo práctico de creación y activación de un entorno virtual con Python:

```
# Creación del entorno virtual
python -m venv venv

# Activación en macOS / Linux
source venv/bin/activate
```

Output visual esperado:

```
(venv) usuario@equipo:~/tu-proyecto$
```

Variables de entorno: por qué los secretos no van al repositorio

Durante este programa vas a usar **credenciales reales**: llaves de **API** de proveedores de modelos, tokens de acceso, y más adelante credenciales de otros servicios.

> 🚫 Ninguna de ellas puede quedar escrita directamente en tu código.



Nunca guardes secretos en el repositorio

El riesgo

La razón es simple: todo lo que subes a tu repositorio queda en el historial para siempre, visible para quien tenga acceso, incluso si después borras esa línea en un **commit** posterior.

Una llave de **API** filtrada en un repositorio público puede ser usada

→

La solución

La solución estándar son las **variables de entorno**: valores que tu programa lee desde el entorno de ejecución en vez de tenerlos escritos en el código, y que se declaran en un archivo que explícitamente se excluye del control de versiones.

por terceros en minutos, generando cargos o abusos a tu nombre.

Vas a configurar esto desde tu primer repositorio, y la exclusión de secretos es uno de los ítems que se revisa en el checkpoint de esta unidad. Si en algún momento del programa commiteas un secreto por error, no basta con borrarlo en el siguiente **commit**: sigue existiendo en el historial, y hay que limpiarlo de ahí también.

> Avísale al equipo docente de inmediato si te pasa, para corregirlo bien.

🛠️ Troubleshooting / Posibles errores



Error: `command not found: python` **en macOS/Linux**

En muchos sistemas basados en UNIX, debes usar el comando **python3** y **pip3** en lugar de `python` o `pip`.



Activación de entornos virtuales en Windows (PowerShell)

Ejecuta `.\venv\Scripts\Activate.ps1`. Si recibes un error de políticas de ejecución, ejecuta PowerShell como administrador y usa: `Set-ExecutionPolicy Unrestricted -Scope Process`.



Comando `git` **no reconocido**

Asegúrate de haber instalado Git y reiniciado tu terminal o VS Code para que reconozca las variables de entorno de tu sistema.



Siguiente lección

Con estos conceptos claros (**VS Code** como editor de referencia, terminal, Git, **entornos virtuales** y **variables de entorno**), en la siguiente lección de este mismo bloque se arma un entorno completo en vivo, de principio a fin, y después lo replicas tú.

Última modificación: jueves, 24 de septiembre de 2026, 10:40